USART Register
(Universal Synchronous/Asynchronous Receiver and Transmitter)

Embedded Controller

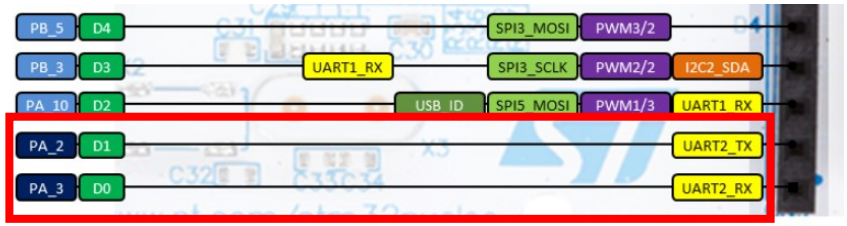
Prof. Young-Keun Kim

Introduction

- **USARTx Configuration**

MCU: STM32F411 RE

- Has 3x USART: USART2, USART1, USART6
- Has 5x SPI: SPI2,SPI3, SPI5, SPI4,SPI1



TX: PA2 RX: PA3

<USART2: COMx port via ST-Link(USB)>

TX/RX pins

Transmit

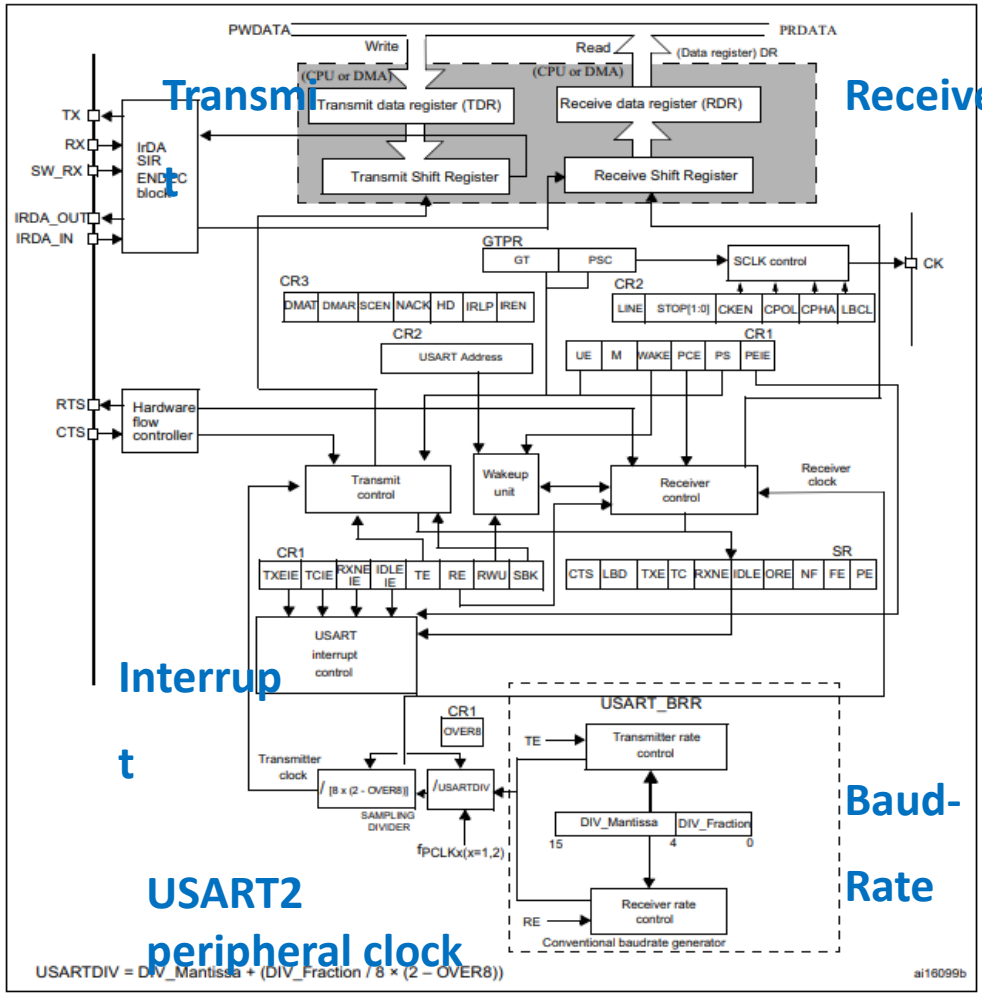
Receive

SPI clo

Interrup t

Baud-Rate

USART2 peripheral clock



USART Register

Overview of USARTx Configuration

Part 1. GPIO AF setting

1. AF mode for GPIOx Pin_y
2. Alternative Function Register for USARTx

Part 2. USART setting (x=2, 1,6)

1. Enable USARTx peripheral clock	**(RCC → APByENR : USARTxEN) //** y=1 if x=2
2. Disable USARTx	**(USARTx → CR1 : UE)**
3. Configure Parity mode , Word Length, Oversampling mode	**(USARTx → CR1 : PCE // M // OVER8)**
4. Set the Stop Bit	**(USARTx → CR2 : STOP)**
5. Configure Baud-Rate(USARTDIV)	**(USARTx → BRR)**
6. Enable Transmitter, Receiver, and USARTx	**(USARTx → CR1 : TE, RE, UE)**

Part 3. Transmit and receive USART data

1. Check the status of USART	**(USARTx → SR : TXE // TC)**
	(USARTx → SR : RXNE)
2. Transmit or receive the data	**(USARTx → DR)**

Part 4. USART Interrupt

Part 1. GPIO_AF Setting

- **GPIO Pin Setting: Alternate Function Output**
  0. Enable GPIO Peripheral Clock (**AHB1ENR**)
  1: Configure as Alternate Function Mode (**GPIOx → MODER**)
  2. GPIO alternate function register (**GPIOx_AFR**)

EX) USART 2
- TX: PA2 RX: PA3

AFRL: PIN 0~7
AFRH: PIN 8~15

8.4.1 GPIO port mode register (GPIOx_MODER) (x = A..E and H)

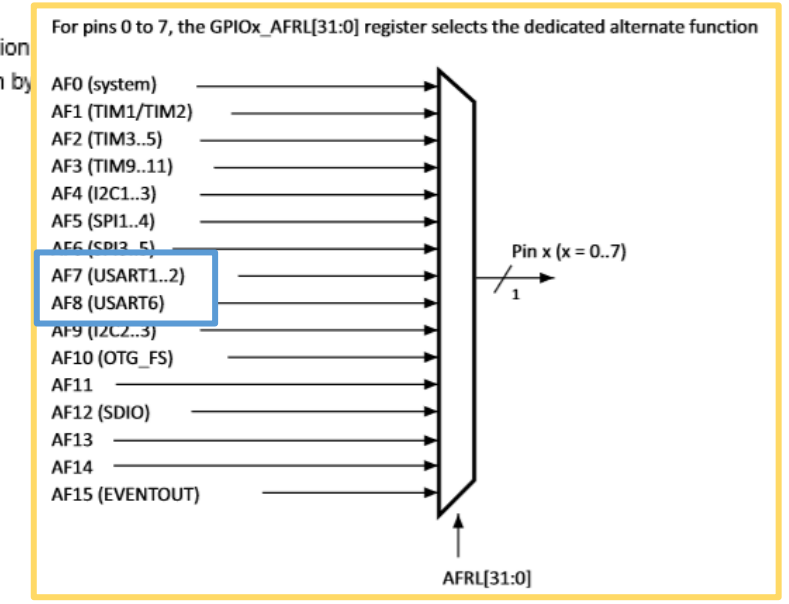
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y:2y+1 **MODERy[1:0]:** Port x configuration bits (y = 0..15)
These bits are written by software to configure the I/O direction mode.
00: Input (reset state)
01: General purpose output mode
10: Alternate function mode
11: Analog mode

8.4.9 GPIO alternate function low register (GPIOx_AFRL) (x = A..E and H)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRL7[3:0]				AFRL6[3:0]				AFRL5[3:0]				AFRL4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRL3[3:0]				AFRL2[3:0]				AFRL1[3:0]				AFRL0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFRLy:** Alternate function
These bits are written by
AFRLy selection:
0000: AF0
0001: AF1
0010: AF2
0011: AF3
0100: AF4
0101: AF5
0110: AF6
0111: AF7



Part 2. USARTx setting

- **Enable USARTx peripheral clock**

1. RCC: Enable USART2 peripheral clock **(RCC → APB1ENR : USART2EN)**

Example: USART2

```
// Enable the clock of USART2
RCC->APB1ENR |= RCC_APB1ENR_USART2EN;
```

6.3.11 RCC APB1 peripheral clock enable register (RCC_APB1ENR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved			PWR EN	Reserved				I2C3 EN	I2C2 EN	I2C1 EN	Reserved			USART2 EN	Reser-ved
			rw					rw	rw	rw				rw	

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 EN	SPI2 EN	Reserved		WWDG EN	Reserved							TIM5 EN	TIM4 EN	TIM3 EN	TIM2 EN
rw	rw			rw								rw	rw	rw	rw

RCC_APB1ENR: USART2, SPI2, SPI3
RCC_APB2ENR: USART1,6, SPI1, SPI4, SPI5

Part 2. USARTx setting

- **Enable/Disable USART2**

2. Disable USART2 **(USART2 → CR1 : UE)**

- Disable before configuration setting

Example: USART2

```
USARTx->CR1 &= ~USART_CR1_UE;
```

19.6.4 Control register 1 (USART_CR1)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
rw	Res.	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 13 **UE**: USART enable

When this bit is cleared, the USART prescalers and outputs are stopped and the end of the current byte transfer in order to reduce power consumption. This bit is set and cleared by software.

0: USART prescaler and outputs disabled

1: USART enabled

Part 2. USARTx setting

● USART Configuration

3. Configure Parity mode , Word Length, Oversampling **(USART2 → CR1 : PCE, M, OVER8)**

19.6.4 Control register 1 (USART_CR1)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
rw	Res.	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 10 **PCE**: Parity control enable

This bit selects the hardware parity control (generation and detection). When the parity control is enabled, the computed parity is inserted at the MSB position (9th bit if M=1; 8th bit if M=0) and parity is checked on the received data. This bit is set and cleared by software. Once it is set, PCE is active after the current byte (in reception and in transmission).

0: Parity control disabled

1: Parity control enabled

Bit 12 **M**: Word length

This bit determines the word length. It is set or cleared by software.

0: 1 Start bit, 8 Data bits, n Stop bit

1: 1 Start bit, 9 Data bits, n Stop bit

Note: The M bit must not be modified during a data transfer (both transmission and reception)

Bit 15 **OVER8**: Oversampling mode

0: oversampling by 16

1: oversampling by 8

Note: Oversampling by 8 is not available in the Smartcard, IrDA and LIN modes: when SCEN=1,IREN=1 or LINEN=1 then OVER8 is forced to '0 by hardware.

Example: USART2

- Use default setting
- No parity, 8-bit word
- Oversample by 16 (default)

```
USARTx->CR1 &= ~USART_CR1_M;
USARTx->CR1 &= ~USART_CR1_PCE;
USARTx->CR1 &= ~USART_CR1_OVER8;
```

Part 2. USARTx setting

● Baud Rate Configuration

4. Configure USARTDIV depending on the baud-rate. (**USARTx → BRR)**
   - 9600, 38400, 57600, 115200

Example: USART2
- 9600 Baud-Rate

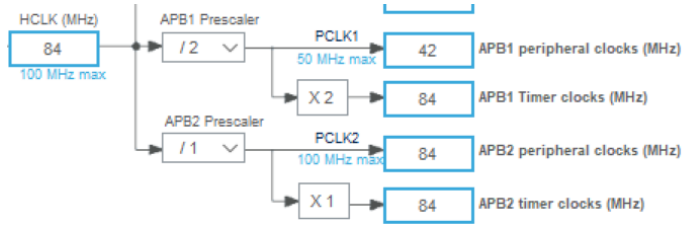
19.6.3 Baud rate register (USART_BRR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIV_Mantissa[11:0]												DIV_Fraction[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Mantissa: USARTDIV
정수 부분

Fraction: USARTDIV
소수점 아래

f_APB1 = CLK/2
f_APB2= CLK/1



$$\text{USARTDIV} = \frac{f_{\text{APBx}}}{16 \times \text{Baud Rate}}$$

$$\text{USARTDIV} = \frac{42{,}000{,}000}{16 \times 9600} = 273.44$$

Mantissa: 273=0x111, Fraction: 0.4*16= (0x7)

[3:0] 하위 4비트가 1/16 단위이기 때문

Part 2. USARTx setting

Baud Rate Configuration

If CLK=84MHz
USART2: f_{APB1} = CLK/2 = 42MHz
USART1,6: f_{APB2} = CLK/1 = 84MHz

Example: Baud Rate = 9600 [bps], for USART2

(Method 1)

$$\text{USARTDIV} = \frac{42{,}000{,}000}{\mathbf{16} \times 9600} = 273.44$$

Mantissa = 273 = 0x111

Fraction = 0.44 x 16 = 0x7

USART_BRR = (Mantissa <<4) | Fraction = 0x1117

```
// Method 1
    float USARTDIV = (float) fCK/16.0/baud;
    uint32_t mantissa = (uint32_t)USARTDIV;
    uint32_t fraction = round((USARTDIV-mantissa)*16);
    USARTx->BRR = ((mantissa<<4)|fraction);
```

(Method 2)

$$\text{BRR} = \frac{42{,}000{,}000}{9600} = 4375$$

* (USARTDIV ≪ 4) = USARTDIVx16

BRR=4375 = 0x1117 = (0x111 << 4)+ 0x7 = 4375

USART_BRR =BRR

```
// Method 2
    uint32_t BRR = (float) fCK/baud;
    USARTx->BRR = BRR;
```

Part 2. USARTx setting

- **Enable TE, RE, UE**

6. Enable Transmitter, Receiver, and USARTx **(USARTx → CR1 : TE, RE, UE)**

```
USARTx->CR1   |= (USART_CR1_RE | USART_CR1_TE);
USARTx->CR1   |= USART_CR1_UE;
```

19.6.4 Control register 1 (USART_CR1)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
rw	Res.	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 3 **TE**: Transmitter enable

This bit enables the transmitter. It is set and cleared by software.

0: Transmitter is disabled

1: Transmitter is enabled

Note: During transmission, a "0" pulse on the TE bit ("0" followed by "1") sends a preamble (idle line) after the current word, except in smartcard mode.

When TE is set, there is a 1 bit-time delay before the transmission starts.

Bit 2 **RE**: Receiver enable

This bit enables the receiver. It is set and cleared by software.

0: Receiver is disabled

1: Receiver is enabled and begins searching for a start bit

Bit 13 **UE**: USART enable

When this bit is cleared, the USART prescalers and outputs are stopped and the end of the current byte transfer in order to reduce power consumption. This bit is set and cleared by software.

0: USART prescaler and outputs disabled

1: USART enabled

Part 3: Transmit and receive data

Transmitting Data

1. (for **TX**): Check the status of USART **(USARTx → SR : TXE , TC)**

- TXE==1 : TRD is transferred to Shift register
- TCE==1 : Transmission is completed

19.6.1 Status register (USART_SR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NF	FE	PE
						rc_w0	rc_w0	r	rc_w0	rc_w0	r	r	r	r	r

Bit 7 **TXE**: Transmit data register empty

This bit is set by hardware when the content of the TDR register has been transferred into the shift register. An interrupt is generated if the TXEIE bit =1 in the USART_CR1 register. It is cleared by a write to the USART_DR register.

0: Data is not transferred to the shift register — TD is NOT empty

1: Data is transferred to the shift register) — TD is empty, ready for next TX

Note: This bit is used during single buffer transmission.

Bit 6 **TC**: Transmission complete

This bit is set by hardware if the transmission of a frame containing data is complete and if TXE is set. An interrupt is generated if TCIE=1 in the USART_CR1 register. It is cleared by a software sequence (a read from the USART_SR register followed by a write to the USART_DR register). The TC bit can also be cleared by writing a '0' to it. This clearing sequence is recommended only for multibuffer communication.

0: Transmission is not complete — TC is still on going

1: Transmission is complete — TC is done. Ready for next TX

```
void USART_write(USART_TypeDef * USARTx, uint8_t *buffer, uint32_t nBytes) {
    int i;

    // TXE is set by hardware when the content of the TDR
    // register has been transferred into the shift register.

    for (i = 0; i < nBytes; i++) {
        // wait until TXE (TX empty) bit is set
        while (!(USARTx->SR & USART_SR_TXE));
        // Writing USART_DR automatically clears the TXE flag
        USARTx->DR = buffer[i] & 0xFF;
        USART_delay(300);
    }
    // wait until TC bit is set
    while (!(USARTx->SR & USART_SR_TC));
    // TC bit clear
    USARTx->SR &= ~USART_SR_TC;
}
```

Part 3: Transmit and receive data

● Receiving Data

2. (for **RX**): Check the status of USART **(USARTx → SR : RXNE)**
- RXNE==1 : Received Data is ready to read

3. Read the data **(USARTx → DR): 8-bit 0xFF**

19.6.1 Status register (USART_SR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NF	FE	PE
						rc_w0	rc_w0	r	rc_w0	rc_w0	r	r	r	r	r

Bit 5 **RXNE**: Read data register not empty

This bit is set by hardware when the content of the RDR shift register has been transferred to the USART_DR register. An interrupt is generated if RXNEIE=1 in the USART_CR1 register. It is cleared by a read to the USART_DR register. The RXNE flag can also be cleared by writing a zero to it. This clearing sequence is recommended only for multibuffer communication.

0: Data is not received

1: Received data is ready to be read. Data 수신 완료

```c
#define USART_SR_RXNE  (0x1UL<< 5U)

uint8_t USART_read(USART_TypeDef * USARTx){
    // Wait until RXNE (RX not empty) bit is set by HW -->Read to read
    while ((USARTx->SR & USART_SR_RXNE) != USART_SR_RXNE);
    // Reading USART_DR automatically clears the RXNE flag
    return ((uint8_t)(USARTx->DR & 0xFF));
}
```

Part 4. USART Receive Interrupt Setting

- **Interrupt USART**
  - For RX: RXNE, ORE
  - For TX: TXE, TC

(For RX Interrupt)

1. Enable RXNE interrupt. **USARTx → CR1 : RXNEIE**
2. NVIC

Control Register 1 (USART CR1)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
rw	Res.	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 5 **RXNEIE**: RXNE interrupt enable
This bit is set and cleared by software.
0: Interrupt is inhibited
1: An USART interrupt is generated whenever ORE=1 or RXNE=1 in the USART_SR register

```
USARTx->CR1 |= USART_CR1_RXNEIE;

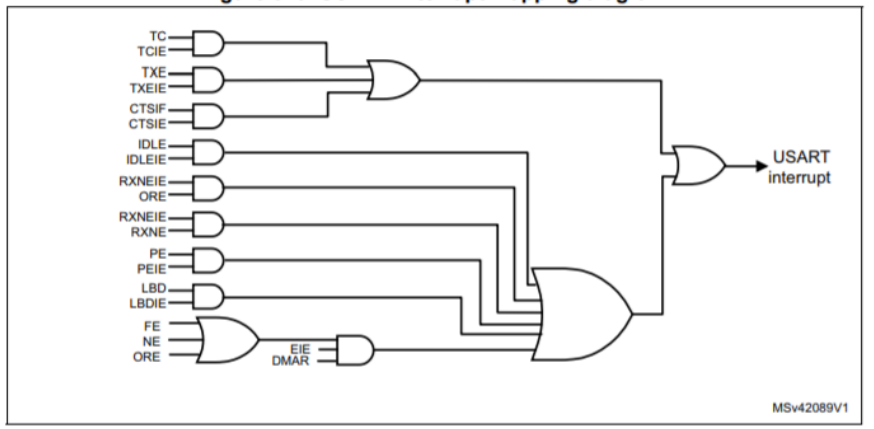
NVIC_SetPriority(USART2_IRQn, 1);    // Set Priority to 1
NVIC_EnableIRQ(USART2_IRQn);         // Enable interrupt of USART2 peripheral
```

30.4 USART interrupts

Table 147. USART interrupt requests

Interrupt event	Event flag	Enable control bit
Transmit Data Register Empty	TXE	TXEIE
CTS flag	CTS	CTSIE
Transmission Complete	TC	TCIE
Received Data Ready to be Read	RXNE	RXNEIE
Overrun Error Detected	ORE	
Idle Line Detected	IDLE	IDLEIE
Parity Error	PE	PEIE
Break Flag	LBD	LBDIE
Noise Flag, Overrun error and Framing Error in multibuffer communication	NF or ORE or FE	EIE

Figure 320. USART interrupt mapping diagram




```
void USART2_IRQHandler(void){
  // User code here---------------

}
```